

Sprint 4: eXtreme Programming (XP) — Estudo de Caso Prático

Projeto: Implementação Prática das Engenharia Ágil no Core do Negócio

Sprint: Sprint 4 (Aplicação dos conceitos)

Data de Entrega: 29/05/2026

1. Resumo do Estudo de Caso

Análise da aplicação real de três práticas fundamentais do eXtreme Programming (XP) em um cenário prático de desenvolvimento de regras corporativas complexas: **Test-Driven Development (TDD)**, **Pair Programming** e **Integração Contínua (CI)**.

2. Detalhamento Técnico das Práticas Implementadas

- **Pair Programming (Programação em Par):** Adotado para codificar a camada crítica de validação de negócios. Enquanto um engenheiro atua como piloto digitando as regras estruturais, o copiloto foca estrategicamente em prever vulnerabilidades, edge cases de domínios e caminhos alternativos do código.
- **Integração Contínua (CI):** Cada alteração de código realizada pelos pares é imediatamente integrada em um trunk principal no GitHub através de Pull Requests rápidos. Um pipeline automatizado executa a validação sintática e rodagem da suite de testes para evitar quebras em produção.

3. Simulação do Fluxo TDD para Regra Complexa de Software

Aplicação do ciclo tático do TDD (Red, Green, Refactor) para validar a criação de regras de elegibilidade avançada em um sistema de cobranças back-end:

```
# Passo 1: Red (Código do Teste Automatizado Base)
import unittest

class TestValidadorFaturamento(unittest.TestCase):
    def test_deve_rejeitar_guia_de_convenio_com_data_vencida(self):
        validador = ValidadorFaturamento()
        guia_invalida = GuiaConvenio(ativa=False, dias_vencidos=10)

        resultado = validador.validar_elegibilidade(guia_invalida)
        self.assertFalse(resultado)

if __name__ == '__main__':
    unittest.main()

# Passo 2: Green (Código Mínimo Escrito para Fazer Passar)
```

```

class GuiaConvenio:
    def __init__(self, ativa: bool, dias_vencidos: int):
        self.ativa = ativa
        self.dias_vencidos = dias_vencidos

class ValidadorFaturamento:
    def validar_elegibilidade(self, guia: GuiaConvenio) -> bool:
        if guia.dias_vencidos > 0:
            return False
        return True

# Passo 3: Refactor (Refatoração para Manutenibilidade e Padrões Limpos)
class GuiaConvenio:
    def __init__(self, ativa: bool, dias_vencidos: int):
        self._ativa = ativa
        self._dias_vencidos = dias_vencidos

    @property
    def esta_expirada(self) -> bool:
        return self._dias_vencidos > 0 or not self._ativa

class ValidadorFaturamento:
    def validar_elegibilidade(self, guia: GuiaConvenio) -> bool:
        return not guia.esta_expirada

```